



4.2. Inserting, Deleting, and Updating Data Values

Previous Section:

 [4.1. Renaming Indexes and Columns](#)

Contents:

[1. Inserting and Deleting Data](#)

[1.1. Adding a New Column](#)

[1.2. Adding a New Row](#)

[1.3. Deleting Columns](#)

[1.4. Deleting Rows](#)

[1.5. Removing a Column with `pop\(\)`](#)

[2. Updating Data Values](#)

[2.1. Updating an Entire Column](#)

[2.2. Updating Specific Rows](#)

[2.3. Updating a DataFrame Using Filtering](#)

[Summary](#)

[References](#)

After organizing a DataFrame with meaningful column names and indexes, the next step is learning how to modify its contents. During data analysis, it is common to add new information, remove unnecessary data, or update existing values as new data becomes available or errors are corrected.

Pandas provides flexible methods for modifying both the **structure** and the **contents** of a DataFrame. You can insert new rows and columns, delete

unwanted data, or update specific values based on conditions. These operations are essential during data cleaning, feature engineering, and dataset maintenance.

In this section, we continue using our synthetic student dataset to demonstrate the most common techniques for inserting, deleting, and updating data.

```
import numpy as np
import pandas as pd

# Reproducible randomness
np.random.seed(42)

names = [
    "James", "Chris", "Adam", "Marian", "Peter",
    "Kevin", "Linda", "Emma", "Sophia", "Daniel",
    "Noah", "Olivia", "Lucas", "Grace", "Henry",
    "Liam", "Ava", "Mia", "Nathan", "Chloe"
]

majors = ["AI", "IT", "CSC", "SW"]

n = 20

df = pd.DataFrame({
    "Name": np.array(names),
    "Age": np.random.randint(18, 21, n),
    "Major": np.random.choice(majors, n),
    "GPA": np.round(np.random.uniform(2.0, 4.0, n), 2)
})

print(df.head(10))
```

1. Inserting and Deleting Data

Pandas allows you to modify the structure of a DataFrame by adding or removing rows and columns.

1.1. Adding a New Column

- The simplest way to add a column is by assigning values to a column name that does not already exist.

```
df["Scholarship"] = False
```

If the column already exists, the assignment updates its values instead of creating a new column.

- You can also assign a list, NumPy array, or Pandas Series, provided its length matches the number of rows.

```
df["Year"] = [1, 2, 3, 4] * 5
```

1.2. Adding a New Row

A new row can be appended using `loc` with the next available index.

```
df.loc[len(df)] = ["Alice", 19, "AI", 3.85]
```

The list (or dictionary) must contain values for every column in the DataFrame.



Note: Pandas also provides the `df.insert()` method, which inserts a column at a specific position in the DataFrame.

```
df.insert(1, "Year", 1)
```

This inserts the `Year` column at column position **1**, shifting the remaining columns to the right.

While `insert()` is useful when the column order matters, I generally recommend using:

```
df["Year"] = 1
```

It is simpler, easier to read, and sufficient for most data analysis tasks. In practice, the exact position of a column rarely affects the analysis, since columns can always be reordered later if needed.

1.3. Deleting Columns

To remove one or more columns, use `drop()`.

```
df.drop(columns=["Major"])
```

To delete multiple columns:

```
df.drop(columns=["Age", "Major", "Scholarship"])
```

By default, `drop()` returns a new DataFrame. To modify the original DataFrame, either assign the result back or specify `inplace=True`.

1.4. Deleting Rows

Rows are removed by specifying their index labels.

```
df.drop(index=0)
```

Delete multiple rows.

```
df.drop(index=[0, 3, 5])
```

Again, this operation returns a new DataFrame unless `inplace=True` is used.

1.5. Removing a Column with `pop()`

The `pop()` method removes a column **in-place** and returns it as a Pandas Series.

```
major = df.pop("Major")
```

Unlike `drop()`, `pop()` immediately modifies the DataFrame while preserving the removed column for later use.



Note: `drop()` can also use the `axis` parameter to specify whether you are deleting rows or columns.

- `axis=0` (default): Delete **rows**.
- `axis=1`: Delete **columns**.

Therefore, the following two statements are equivalent:

```
df.drop(columns=["Major"])
```

```
df.drop(["Major"], axis=1)
```

The `columns=` and `index=` parameters are generally more readable and are recommended over using `axis`.

2. Updating Data Values

Besides changing the DataFrame's structure, we often need to modify existing values.

2.1. Updating an Entire Column

Assigning a value to a column replaces every value in that column.

```
df["Major"] = "CSC"
```

Output: All students now have Major = CSC.

You can also assign a list or Series.

```
df["Age"] = np.random.randint(18, 21, len(df))
```

2.2. Updating Specific Rows

Often we only want to update rows satisfying a condition.

For example, award a scholarship to students with a GPA of at least **3.5**.

```
df["Scholarship"] = False  
df.loc[df["GPA"] >= 3.5, "Scholarship"] = True
```

The syntax is `df.loc[filtering_condition, "column_name"] = value`

Only rows satisfying the filtering condition are modified.

2.3. Updating a DataFrame Using Filtering

Filtering is commonly used to create a new DataFrame containing only selected rows.

```
high_gpa = df[df["GPA"] >= 3.5]
```

Only students with GPA ≥ 3.5 remain.

Unlike `loc`, this does **not** modify the original DataFrame. Instead, it creates a new filtered DataFrame.

Summary

DataFrames can be modified by inserting, deleting, or updating rows and columns.

- `df["column_name"] = value` creates a new column if it does not already exist; otherwise, it updates the existing column.

- `df.loc[len(df)] = data` appends a new row to the end of the DataFrame.
- `df.drop(columns=...)` removes columns, while `df.drop(index=...)` removes rows.
- `df.pop("column_name")` removes a column in-place and returns it as a Pandas Series.
- Assigning a value to an entire column replaces every value in that column.
- `df.loc[filtering_condition, "column_name"] = value` updates only the rows satisfying the specified condition.
- `df = df[filtering_condition]` creates a new filtered DataFrame and is commonly used to retain only the rows of interest.

To practice further, complete Lab 04 of the Data Analytics Hands-On Lab.

 [Lab 07: Data Analytics with Python](#)

References

<https://www.geeksforgeeks.org/pandas/dealing-with-rows-and-columns-in-pandas-dataframe/>

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.insert.html>

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.drop.html>

Next Section:

 [5.1. Data Grouping and Aggregation](#)